LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 4 & 5
ENCAPSULATION

Disusun Oleh:

Iman Dwi Satrio 105224029

PROGRAM STUDI ILMU KOMPUTER

FAKULTAS SAINS DAN KOMPUTER

UNIVERSITAS PERTAMINA

2026

I. Pendahuluan

Praktikum Modul 4 dan 5 ini berfokus pada dua hal penting dalam Object-Oriented Programming (OOP) yaitu Encapsulation dan Method Overloading. Encapsulation memungkinkan developer untuk melindungi data sensitif di dalam kelas dengan menggunakan akses modifier private dan menyediakan kontrol akses melalui metode getter dan setter. Hal ini berguna untuk memastikan integritas data melalui proses validasi sebelum data disimpan ke dalam atribut objek. Modul ini juga mengimplementasikan Method Overloading di mana sebuah kelas dapat memiliki beberapa metode dengan nama yang sama namun berbeda dalam parameter (jumlah atau tipe data). Implementasi konsep ini diwujudkan dalam sistem manajemen Kamar Hotel.

II. Variabel

No	Nama Variabel	Tipe data	Fungsi
1	nomorKamar	String	Menyimpan identitas unik nomor kamar hotel.
2	tipeKamar	String	Menyimpan kategori kamar (Reguler, Premium, Suite).
3	kapasitasMaksimal	int	Menyimpan batas maksimal jumlah tamu yang diperbolehkan di satu kamar.
4	hargaPerMalam	double	Menyimpan nilai nominal biaya sewa kamar untuk satu malam.
5	isTersedia	boolean	Penanda status ketersediaan kamar (true jika kosong).
6	tipe	String	Parameter input pada setter untuk validasi kategori kamar.
7	harga	double	Parameter input pada setter untuk validasi batas harga minimal.
8	jumlahTamu	int	Parameter untuk mengecek kecukupan kapasitas kamar saat dipesan.
9	jumlahMalam	int	Penentu durasi inap untuk perhitungan total bayar.
10	kodeVoucher	String	Parameter input untuk klaim diskon harga.
11	totalBayar	double	Penampung hasil kalkulasi biaya akhir sebelum diskon.

12	kamar1	KamarHotel	Objek yang dibuat untuk menyimpan objek kamar 1 di Main.
13	kamar2	KamarHotel	Objek yang dibuat untuk menyimpan objek kamar 2 di Main.
14	tagihanKamar1	double	Menyimpan hasil dari hitung total bayar atau tagihan objek kamar 1
15	tagihanKamar2	double	Menyimpan hasil dari hitung total bayar atau tagihan objek kamar 2

III. Constructor dan Method

No	Nama Metode	Jenis Metode	Fungsi
1	KamarHotel (1)	Constructor	Inisialisasi objek kamar dengan pendaftaran kilat (harga default 0).
2	KamarHotel (2)	Constructor	Inisialisasi objek dengan data lengkap termasuk harga per malam.
3	getNomorKamar	Functional	Mengambil nilai dari atribut private nomorKamar (Getter).
4	getTipeKamar	Functional	Mengambil nilai dari atribut private tipeKamar (Getter).
5	getKapasitasMaksimal	Functional	Mengambil nilai dari atribut private kapasitasMaksimal (Getter).
6	getHargaPerMalam	Functional	Mengambil nilai dari atribut private hargaPerMalam (Getter).
7	getIsTersedia	Functional	Mengambil nilai dari atribut private isTersedia (Getter).
8	setTipeKamar	Functional	Mengubah tipe kamar dengan validasi kategori tertentu.
9	setHargaPerMalam	Functional	Mengubah harga kamar dengan validasi batas minimum Rp50.000.
10	pesanKamar()	Procedural	Melakukan pemesanan kamar tanpa memandang jumlah tamu.

11	pesanKamar(int)	Procedural	Melakukan pemesanan dengan validasi kapasitas maksimal tamu.
12	batalPesan	Procedural	Mengubah status kamar kembali menjadi tersedia.
13	hitungTotalBayar(int)	Functional	Menghitung biaya inap normal berdasarkan durasi malam.
14	hitungTotalBayar(int, string)	Functional	Menghitung biaya inap dengan validasi diskon voucher PROMO.
15	infoKamar	Procedural	Menampilkan profil lengkap informasi kamar ke layar.

IV. Dokumentasi dan Pembahasan Code

```
public class KamarHotel {
   private String nomorKamar;
   private String tipeKamar;
   private int kapasitasMaksimal;
   private double hargaPerMalam;
   private boolean isTersedia;

   // Constructor 1 Pendaftaran Kilat
            public KamarHotel(String nomorKamar, String tipeKamar, int
kapasitasMaksimal) {
      this.nomorKamar = nomorKamar;
      setTipeKamar(tipeKamar); // Menggunakan setter
      this.kapasitasMaksimal = kapasitasMaksimal;
      this.hargaPerMalam = 0; // sistem akan otomatis mengatur harga menjadi 0
         this.isTersedia = true; // sistem akan otomatis mengatur status isTersedia
menjadi true
   }

   // Constructor 2 Pendaftaran Lengkap
            public KamarHotel(String nomorKamar, String tipeKamar, int
kapasitasMaksimal, double hargaPerMalam) {
      this.nomorKamar = nomorKamar;
      setTipeKamar(tipeKamar); // Menggunakan setter
      this.kapasitasMaksimal = kapasitasMaksimal;
      setHargaPerMalam(hargaPerMalam); // Menggunakan setter
         this.isTersedia = true; // status ketersediaan yang juga langsung diatur true
secara bawaan
   }
```

```
    // Getter
    public String getNomorKamar() {
        return nomorKamar;
    }
    public String getTipeKamar() {
        return tipeKamar;
    }
    public int getKapasitasMaksimal() {
        return kapasitasMaksimal;
    }
    public double getHargaPerMalam() {
        return hargaPerMalam;
    }
    public boolean isTersedia() {
        return isTersedia;
    }

    // Setter
    public void setTipeKamar(String tipe) {
          if (tipe.equalsIgnoreCase("Reguler") || tipe.equalsIgnoreCase("Premium") ||
tipe.equalsIgnoreCase("Suite")) {
            this.tipeKamar = tipe;
        } else {
              System.out.println("[PERINGATAN] Tipe Kamar " + tipe + " tidak ada.
Otomatis diatur ke: Reguler");
            this.tipeKamar = "Reguler";
        }
    }

    public void setHargaPerMalam(double harga) {
        if (harga < 50000) {
                System.out.println("[PERINGATAN] Harga Rp" + harga + " ditolak.
Minimum diatur ke: Rp. 50000");
            this.hargaPerMalam = 50000;
        } else {
            this.hargaPerMalam = harga;
        }
    }

    // Method Overloading: Pemesanan
    public void pesanKamar() {
        if (isTersedia) {
            isTersedia = false;
                System.out.println("[BERHASIL] Kamar " + nomorKamar + " telah
dipesan.");
```

```
        } else {
            System.out.println("[GAGAL] Kamar " + nomorKamar + " sudah terisi.");
        }
    }

    public void pesanKamar(int jumlahTamu) {
        if (!isTersedia) {
            System.out.println("[GAGAL] Kamar " + nomorKamar + " sudah terisi.");
        } else if (jumlahTamu > kapasitasMaksimal) {
            System.out.println("[GAGAL] Kamar ini maksimal " + kapasitasMaksimal
+ " orang.");
        } else {
            isTersedia = false;
                System.out.println("[BERHASIL] Kamar " + nomorKamar + " dipesan
untuk " + jumlahTamu + " tamu.");
        }
    }

    public void batalPesan() {
        isTersedia = true;
           System.out.println("[BERHASIL] Status Kamar " + nomorKamar + " kini
Tersedia.");
    }

    // Method Overloading: Penagihan
    public double hitungTotalBayar(int jumlahMalam) {
        return hargaPerMalam * jumlahMalam;
    }

    public double hitungTotalBayar(int jumlahMalam, String kodeVoucher) {
        double totalBayar = hitungTotalBayar(jumlahMalam);
        if (kodeVoucher.equalsIgnoreCase("PROMO") && jumlahMalam >= 3) {
            System.out.println("[BERHASIL] Anda mendapatkan diskon 20%.");
            return totalBayar * 0.8;
        } else {
              System.out.println("[GAGAL] Syarat diskon tidak terpenuhi. Anda pakai
harga normal.");
            return totalBayar;
        }
    }

    public void infoKamar() {
        System.out.println("-- PROFIL KAMAR --");
        System.out.println("Nomor Kamar: " + nomorKamar);
        System.out.println("Tipe: " + tipeKamar);
        System.out.println("Kapasitas Max: " + kapasitasMaksimal + " orang");
```

```
        System.out.println("Harga/Malam: Rp" + hargaPerMalam);
        System.out.println("Status: " + (isTersedia ? "Tersedia" : "Tidak Tersedia"));
    }
}
```

Bagian ini akan membahas logika pemrograman yang diimplementasikan. Program ini dirancang untuk menangani sistem transaksi tiket konser secara mandiri.

```java
public class KamarHotel {
    private String nomorKamar;
    private String tipeKamar;
    private int kapasitasMaksimal;
    private double hargaPerMalam;
    private boolean isTersedia;
```

Penjelasan: Kode di atas adalah deklarasi kelas KamarHotel beserta atribut-atribut pendukungnya yang menerapkan prinsip Encapsulation dalam pemrograman berorientasi objek. Seluruh atribut, mulai dari nomorKamar hingga isTersedia, dideklarasikan menggunakan akses modifier private untuk menjalankan fungsi penyembunyian data dari akses luar secara langsung sehingga risiko masuknya data yang tidak logis ke dalam sistem dapat dicegah sejak tahap awal pendeklarasian.

```java
// Constructor 1 Pendaftaran Kilat
 public KamarHotel(String nomorKamar, String tipeKamar, int kapasitasMaksimal) {
     this.nomorKamar = nomorKamar;
     setTipeKamar(tipeKamar); // Menggunakan setter
     this.kapasitasMaksimal = kapasitasMaksimal;
     this.hargaPerMalam = 0; // sistem akan otomatis mengatur harga menjadi 0
     this.isTersedia = true; // sistem akan otomatis mengatur status isTersedia menjadi true
 }

 // Constructor 2 Pendaftaran Lengkap
 public KamarHotel(String nomorKamar, String tipeKamar, int kapasitasMaksimal, double hargaPerMalam) {
     this.nomorKamar = nomorKamar;
     setTipeKamar(tipeKamar); // Menggunakan setter
     this.kapasitasMaksimal = kapasitasMaksimal;
     setHargaPerMalam(hargaPerMalam); // Menggunakan setter
     this.isTersedia = true; // status ketersediaan yang juga langsung diatur true secara bawaan
 }
```

Penjelasan: Kode tersebut merupakan implementasi Constructor Overloading di mana terdapat dua buah metode konstruktor dengan parameter yang berbeda untuk memberikan fleksibilitas dalam pembuatan objek. Konstruktor pertama (Pendaftaran Kilat) untuk inisialisasi kamar hanya dengan memasukkan data nomor, tipe, dan kapasitas, dan sistem secara otomatis mengatur harga ke nilai nol dan status ketersediaan ke posisi aktif (true). Konstruktor kedua (Pendaftaran

Lengkap) untuk pengguna yang ingin menginput nilai harga per malam secara manual saat objek pertama kali dibuat. Penggunaan metode Setter (seperti setTipeKamar dan setHargaPerMalam) di dalam badan konstruktor dilakukan agar nilai yang dimasukkan oleh pengguna tetap harus melalui proses validasi terlebih dahulu, sehingga integritas data objek terjaga sejak awal proses inisialisasi.

```java
// Getter
 public String getNomorKamar() {
     return nomorKamar;
 }
 public String getTipeKamar() {
     return tipeKamar;
 }
 public int getKapasitasMaksimal() {
     return kapasitasMaksimal;
 }
 public double getHargaPerMalam() {
     return hargaPerMalam;
 }
 public boolean isTersedia() {
     return isTersedia;
 }
```

Penjelasan: Bagian kode ini berisi kumpulan metode Getter yang berfungsi sebagai pintu akses keluar bagi atribut-atribut yang telah diproteksi dengan modifier private. Melalui metode ini, kelas lain dapat mengambil atau membaca nilai data seperti nomor kamar, tipe, hingga harga tanpa bisa mengubahnya secara langsung sehingga prinsip read-only terjaga.

```java
// Setter
public void setTipeKamar(String tipe) {
    if (tipe.equalsIgnoreCase("Reguler") || tipe.equalsIgnoreCase("Premium") || tipe.equalsIgnoreCase("Suite")) {
        this.tipeKamar = tipe;
    } else {
        System.out.println("[PERINGATAN] Tipe Kamar " + tipe + " tidak ada. Otomatis diatur ke: Reguler");
        this.tipeKamar = "Reguler";
    }
}
```

Penjelasan: Bagian ini mengimplementasikan metode Setter untuk atribut tipeKamar yang dilengkapi dengan logika validasi untuk menjaga integritas data. Penggunaan struktur pengkondisian if-else dan metode equalsIgnoreCase digunakan agar sistem bisa memastikan bahwa hanya tipe kamar yang valid yaitu "Reguler", "Premium", atau "Suite" yang dapat disimpan ke dalam objek tanpa mempedulikan penggunaan huruf besar atau kecil. Jika input yang diberikan pengguna tidak sesuai dengan kategori tersebut, sistem akan memberikan pesan peringatan dan secara otomatis menetapkan tipe kamar ke nilai bawaan (default) yaitu "Reguler".

```java
public void setHargaPerMalam(double harga) {
    if (harga < 50000) {
        System.out.println("[PERINGATAN] Harga Rp" + harga + " ditolak. Minimum diatur ke: Rp. 50000");
        this.hargaPerMalam = 50000;
    } else {
        this.hargaPerMalam = harga;
    }
}
```

Penjelasan: Metode setHargaPerMalam ini berfungsi sebagai pengontrol validasi untuk memastikan bahwa nilai biaya sewa kamar tidak berada di bawah standar operasional yang ditetapkan. Sistem akan membatasi harga minimal sebesar Rp50.000 sehingga jika input yang dimasukkan lebih rendah dari batas tersebut, program akan menampilkan pesan peringatan dan secara otomatis mengoreksi nilainya ke batas minimum tersebut.

```java
// Method Overloading: Pemesanan
public void pesanKamar() {
    if (isTersedia) {
        isTersedia = false;
        System.out.println("[BERHASIL] Kamar " + nomorKamar + " telah dipesan.");
    } else {
        System.out.println("[GAGAL] Kamar " + nomorKamar + " sudah terisi.");
    }
}

public void pesanKamar(int jumlahTamu) {
    if (!isTersedia) {
        System.out.println("[GAGAL] Kamar " + nomorKamar + " sudah terisi.");
    } else if (jumlahTamu > kapasitasMaksimal) {
        System.out.println("[GAGAL] Kamar ini maksimal " + kapasitasMaksimal + " orang.");
    } else {
        isTersedia = false;
        System.out.println("[BERHASIL] Kamar " + nomorKamar + " dipesan untuk " + jumlahTamu + " tamu.");
    }
}
```

Penjelasan: Bagian ini mengimplementasikan konsep Method Overloading pada metode pesanKamar yang memberikan dua cara berbeda untuk memproses pemesanan kamar dalam satu kelas. Metode pertama tidak menggunakan parameter dan hanya berfokus pada pengecekan status ketersediaan secara umum. Jika kamar kosong, statusnya akan langsung diubah menjadi terisi. Sedangkan metode kedua menyertakan parameter jumlahTamu untuk menjalankan logika bisnis yang memvalidasi apakah jumlah orang yang akan menginap melebihi kapasitasMaksimal kamar tersebut.

```java
public void batalPesan() {
    isTersedia = true;
    System.out.println("[BERHASIL] Status Kamar " + nomorKamar + " kini Tersedia.");
}
```

Penjelasan: Metode batalPesan ini berfungsi untuk membatalkan status pesanan pada sebuah kamar dengan cara mengubah kembali nilai atribut isTersedia menjadi true.

```java
// Method Overloading: Penagihan
public double hitungTotalBayar(int jumlahMalam) {
    return hargaPerMalam * jumlahMalam;
}

public double hitungTotalBayar(int jumlahMalam, String kodeVoucher) {
    double totalBayar = hitungTotalBayar(jumlahMalam);
    if (kodeVoucher.equalsIgnoreCase("PROMO") && jumlahMalam >= 3) {
        System.out.println("[BERHASIL] Anda mendapatkan diskon 20%.");
        return totalBayar * 0.8;
    } else {
        System.out.println("[GAGAL] Syarat diskon tidak terpenuhi. Anda pakai harga normal.");
        return totalBayar;
    }
}
```

Penjelasan: Kode di atas menerapkan konsep Method Overloading pada fungsi penagihan untuk memberi fleksibilitas dalam perhitungan biaya menginap tamu. Metode pertama berfungsi untuk menghitung total tagihan secara standar berdasarkan durasi menginap dikalikan dengan harga per malam. Sedangkan metode kedua menambahkan parameter kodeVoucher untuk menangani logika promo. Di dalam metode kedua, terdapat validasi di mana diskon sebesar 20% hanya akan diberikan jika pengguna memasukkan kode "PROMO" dan memenuhi syarat durasi menginap minimal tiga malam. Penggunaan metode equalsIgnoreCase memastikan input voucher tidak peka terhadap huruf besar-kecil.

```java
public void infoKamar() {
    System.out.println("-- PROFIL KAMAR --");
    System.out.println("Nomor Kamar: " + nomorKamar);
    System.out.println("Tipe: " + tipeKamar);
    System.out.println("Kapasitas Max: " + kapasitasMaksimal + " orang");
    System.out.println("Harga/Malam: Rp" + hargaPerMalam);
    System.out.println("Status: " + (isTersedia ? "Tersedia" : "Tidak Tersedia"));
}
```

Penjelasan: Metode infoKamar ini berfungsi sebagai fungsi untuk menampilkan ringkasan profil lengkap dari sebuah objek kamar secara terstruktur ke layar konsol. Melalui metode ini, pengguna dapat melihat status terkini dari seluruh atribut privat, mulai dari nomor kamar, tipe, kapasitas maksimal, hingga harga per malam.

V. Kesimpulan

Kesimpulan dari praktikum Modul 4 dan 5 ini adalah bahwa Encapsulation penting untuk menjaga integritas data melalui validasi pada metode setter, sehingga objek tidak memiliki nilai yang tidak logis. Selain itu, penggunaan Method Overloading memberikan fleksibilitas pada program untuk menangani berbagai skenario input (seperti pemesanan dengan atau tanpa detail jumlah tamu) tanpa harus membuat nama metode yang berbeda-beda, sehingga kode menjadi lebih rapi dan mudah dipahami.

VI. Daftar Pustaka

Deitel, P., & Deitel, H. (2017). Java How to Program, Early Objects. 11th Edition. New York: Pearson Education.

Liang, Y. D. (2020). Introduction to Java Programming and Data Structures. 12th Edition. New York: Pearson.

Schildt, H. (2018). Java: The Complete Reference. 11th Edition. New York: McGraw-Hill Education.